

Secure P2P File Transfer — Interview Defense Document

Prepared strictly from your source: `cnft_protocol.py`, `transfer_resume.py`,
`transfer_security.py`, `work_rec.py`, `work_sender.py`, `receiver_detection.py`

SECTION 1 — 30-Second Pitch

"I built a peer-to-peer encrypted file transfer tool in Python — no relay server, the sender connects directly to the receiver over TCP. Files are split into 4MB chunks, each encrypted independently with AES-256 in EAX mode, so I get both confidentiality and per-chunk tamper detection from the authentication tag. The encryption key isn't the password itself — I run the password through PBKDF2 with 200,000 rounds and a random salt to get a session key, then derive a unique per-chunk key from that using HMAC-SHA256, so no two chunks ever reuse key material. Before any file data moves, both sides do a challenge-response handshake using a shared preset code, so a random device on the network can't just connect and start pulling files. On top of that I built my own resume protocol: the receiver keeps a bitmap of which chunks it already has on disk, so if the connection drops, the sender only re-sends what's missing — verified by re-hashing the whole file with SHA-256 at the end. I also added replay protection using session IDs with a freshness window, and IP-based rate limiting on failed auth attempts to blunt brute-forcing."

This is accurate to your code. Don't add "TLS," "end-to-end encryption between multiple hops," or "production-grade" — none of that is implemented or claimed by your files.

SECTION 2 — Complete Architecture (Stage by Stage)

Stage	Implemented?	Where / How
Connection establishment	✓	Sender opens a raw <code>socket.socket(AF_INET, SOCK_STREAM)</code> and calls <code>client.connect((server_ip, server_port))</code> — a new socket per chunk , not one persistent connection (see <code>send_chunk</code> in <code>work_sender.py</code>).
Authentication	✓	Raw-byte HMAC-SHA256 challenge-response using the shared <code>preset_code</code> , in <code>authenticate_sender</code> (receiver) / <code>authenticate_with_receiver</code> (sender). Happens before any CNFT header is sent.
Rate limiting on auth	✓	<code>transfer_security.can_attempt_auth</code> — 5 failures in 5 minutes → 10-minute block, keyed by peer IP, persisted to <code>.p2p_security/auth_throttle.json</code> .
Key derivation	✓	<code>derive_session_key</code> : PBKDF2(password, salt, dkLen=32, count=200_000) → 256-bit session key. <code>derive_chunk_key</code> : HMAC-SHA256(session_key, <code>"CNFT/1.0-chunk-key" + chunk_id</code>) → per-chunk key (HKDF-Expand <i>style</i> , not full RFC 5869 HKDF — no extract phase).
Encryption	✓	AES-256-EAX per chunk, fresh random 16-byte nonce per chunk (<code>get_random_bytes(16)</code>), producing ciphertext + 16-byte auth tag.
Session/replay protocol layer	✓	<code>MSG_RESUME_QUERY</code> / <code>MSG_DATA</code> sent via <code>cnft_protocol.send_header</code> , followed by a custom binary metadata block (<code>send_transfer_metadata</code>) — filename, size, chunk size, transfer_id, session_id, timestamp, file hash, salt.
File transmission	✓	Each missing chunk sent over its own connection in a <code>ThreadPoolExecutor</code> (up to 8 workers), via <code>send_chunk</code> .
Integrity verification	✓	Two layers: (1) AES-EAX tag verified per chunk via <code>decrypt_and_verify</code> — tampering raises immediately; (2) whole-file SHA-256 recomputed and compared after reassembly, in <code>_verify_file_integrity</code> .

Stage	Implemented?	Where / How
Error handling	⚠️ Partial	Generic <code>try/except</code> around connection/auth errors; <code>cnft_protocol.send_error</code> / <code>MSG_ERROR</code> exist but are never called or imported anywhere in <code>work_rec.py</code> / <code>work_sender.py</code> — this is dead code today.
Transfer completion	✅	<code>bitmap_all_set</code> triggers <code>os.replace(part_path, final_path)</code> , <code>finalize_manifest</code> , <code>mark_session_completed</code> (closes replay window), then hash verification and a completion print.

Not present anywhere in your code: TLS/transport encryption of the handshake itself, public-key exchange, authorization/access-control beyond the shared preset code, encrypted-at-rest storage, or CLI flags (everything is interactive `input()`).

SECTION 3 — File-by-File Walkthrough

`cnft_protocol.py`

- **Purpose:** Thin framing layer — every post-connect message starts with an ASCII line `"CNFT/1.0 <TYPE>\n"`.
- **Functions:**
 - `send_header(sock, msg_type)` — encodes and sends the header line.
 - `recv_header(sock)` — reads byte-by-byte until `\n`, enforces a 64-byte max length, splits on the first space, validates the version string. Raises `ValueError` on malformed input, `ConnectionError` if the socket closes mid-read.
 - `send_error(sock, message)` — sends an `ERROR` header + length-prefixed UTF-8 message. **Unused in the actual transfer flow.**
- **Constants:** `MSG_AUTH`, `MSG_METADATA`, `MSG_DATA`, `MSG_ACK`, `MSG_ERROR`, `MSG_RESUME_QUERY`, `MSG_RESUME_BITMAP` — only `MSG_DATA`, `MSG_ACK`, `MSG_RESUME_QUERY`, `MSG_RESUME_BITMAP` are actually used elsewhere.
- **Depends on:** nothing but a raw socket. **Used by:** `work_rec.py`, `work_sender.py`.

`transfer_resume.py`

- **Purpose:** Everything about chunking, resumability, and cryptographic key derivation for chunks.
- **Key functions:**

- `recv_exact` / `send_uint32` / `recv_uint32` / `send_uint64` / `recv_uint64` / `send_blob` / `recv_blob` — low-level framed I/O primitives (length-prefixed blobs, fixed-width integers).
- `send_transfer_metadata` / `recv_transfer_metadata` — serializes/deserializes the per-transfer metadata struct (filename, size, chunk_size, total_chunks, transfer_id, session_id, timestamp, file hash, salt).
- `chunk_count`, `chunk_offset`, `chunk_size_for_id` — pure math for splitting a file into 4MB (`DEFAULT_CHUNK_SIZE`) pieces, last chunk sized to the remainder.
- `build_transfer_id` — SHA-256 hash of `abspath|file_size|mtime_ns|chunk_size`, truncated to 32 hex chars. **Inference:** because this is keyed on the sender's absolute path and mtime, resuming only works if you resend the *exact same file from the exact same path* — a copy or a touched file gets a new `transfer_id` and starts over.
- `empty_bitmap`, `bitmap_has_chunk`, `bitmap_mark_chunk`, `bitmap_all_set`, `list_missing_chunks` — bit-level manifest tracking (1 bit per chunk).
- `received_bytes_from_bitmap` — sums bytes for progress-bar math.
- `load_manifest` / `save_manifest` / `build_manifest` / `ensure_manifest` — JSON manifest persistence under `.p2p_resume/<transfer_id>.json`; `ensure_manifest` also validates that a re-attached manifest actually matches the current transfer's filename/size/chunk_size (defends against ID collisions).
- `ensure_part_file` — pre-allocates the output file to full size via `truncate()` so chunks can be written at arbitrary offsets even out of order.
- `finalize_manifest` — marks manifest complete after the last chunk lands.
- `compute_file_hash` — streaming SHA-256 with a `tqdm` progress bar.
- `derive_session_key` / `derive_chunk_key` — the two-tier KDF described in Section 2.
- **Depends on:** `hashlib`, `hmac`, `struct`, `pycryptodome`'s `PBKDF2`, `tqdm`. **Used by:** both sender and receiver.

`transfer_security.py`

- **Purpose:** Two independent JSON-file-backed defenses — auth-attempt throttling and session replay protection.
- **Key functions:**
 - `generate_session_id` — `secrets.token_hex(16)` (cryptographically random 32-hex-char ID).
 - `can_attempt_auth` / `record_auth_failure` / `record_auth_success` / `cleanup_auth_throttle_cache` — sliding-window (5 min) failure counter per peer IP; 5 failures → blocked for 10 minutes. Persisted to `.p2p_security/auth_throttle.json`, guarded by a module-level `threading.Lock`.

- `session_fingerprint` — joins `transfer_id/filename/size/chunk_size/total_chunks` into one string, used to detect a `session_id` being reused for a *different* transfer (rejected as `"duplicate session id for different transfer"`).
- `is_timestamp_fresh` — rejects session timestamps more than 15 minutes (`SESSION_FRESHNESS_WINDOW_SECONDS`) from "now" in either direction.
- `register_or_reject_session` — the actual replay gate: fresh timestamp required, fingerprint must match on reuse, and a session already marked `"completed"` is rejected outright.
- `mark_session_completed` / `is_replayed_session` — close out a session after a full file lands; completed sessions are retained 24h (`SESSION_COMPLETED_RETENTION_SECONDS`) then pruned.
- **Depends on:** `json`, `secrets`, `threading`. **Used by:** `work_rec.py` only (the receiver is the one enforcing these checks — sender just generates the `session_id/timestamp`).

`work_rec.py` (Receiver)

- **Purpose:** The server. Listens, authenticates, dispatches by message type, writes chunks, verifies integrity.
- **Global state (module-level, guarded by `lock`):** `password_global`, `session_key_global`, `session_salt_global`, `active_transfer_id`, `last_activity_ts`, `transfer_completed` — reset per file via `reset_state()`.
- **Key functions:**
 - `get_preset_code` — interactive prompt, no `.env` file yet (your own `note.txt` flags this as a TODO).
 - `authenticate_sender` — the receiver's half of the challenge-response (see Section 5 for the exact flow — it's important and slightly unusual).
 - `_prompt_password` / `_cache_session_key` / `_get_session_key` — ensures the operator is asked for the password **once per file**, even though dozens of chunk-connections happen.
 - `_verify_file_integrity` — final SHA-256 comparison, prints  / .
 - `handle_resume_query` — services `MSG_RESUME_QUERY`: registers the session, loads/creates the manifest, derives the session key, and replies with the current bitmap.
 - `handle_chunk_data` — services `MSG_DATA`: validates `chunk_id` range, decrypts+verifies the AES-EAX tag, writes to the pre-allocated part file at the correct offset, updates the bitmap, and finalizes the file when complete — **the whole decrypt+write+bitmap-update critical section runs inside the global `lock`**, so it's

serialized across threads even though multiple sockets can be accepted concurrently.

- `receive_connection` — per-connection worker: peer-IP throttle check → auth → header dispatch → error catch-all.
- `receive_one_file` — accept loop with a `socket.timeout(2)` poll and an 8-second idle-grace shutdown once a transfer has gone quiet.
- `start_server` — binds, listens (backlog 50), loops `receive_one_file`, offers to receive another file.
- **Depends on:** `cnft_protocol`, `transfer_resume`, `transfer_security`, `receiver_detection`, `pycryptodome`. **Interacts with:** every other module in the project.

`work_sender.py` (Client)

- **Purpose:** Picks a file, authenticates, requests the resume bitmap, then fans out missing chunks across a thread pool with per-chunk retries.
- **Key functions:**
 - `authenticate_with_receiver` — sender's half of the challenge-response.
 - `request_resume_bitmap` — one dedicated connection that sends `MSG_RESUME_QUERY` + metadata and gets back the receiver's current bitmap.
 - `send_chunk` — the workhorse: new connection, auth, send `MSG_DATA` + metadata + `chunk_id`, read/encrypt the chunk, send nonce+tag+ciphertext, wait for `MSG_ACK`. Retries up to `MAX_RETRIES=3` with `0.5*attempt`-second backoff on connection-level errors; a non-connection exception breaks immediately (no retry).
 - `send_file` — orchestrates: hash the whole file, derive salt/session key, get the bitmap, submit all missing chunks to a `ThreadPoolExecutor(max_workers=min(8, total_chunks))`, and if any chunks fail, re-query the bitmap and retry up to `MAX_RESUME_ROUNDS=3` rounds.
 - `pick_file` — spawns an isolated `tkinter` subprocess for a native file picker (avoids Tk/thread conflicts).
- **Depends on:** same crypto/protocol modules as the receiver, plus `concurrent.futures`.

`receiver_detection.py`

- **Purpose:** Cross-platform "what's my LAN IP" helper for the auto-fill prompt.
 - **Logic:** branches on `platform.system()` — Windows uses `ipconfig` + regex on `IPv4 Address`; macOS tries `ipconfig getifaddr en0` first, falls back to parsing `ifconfig`; Linux tries `hostname -I`, falls back to parsing `ip addr`. Universal last resort: `socket.gethostbyname(socket.gethostname())`. Filters out anything starting with `127.`.
-

SECTION 4 — Networking, Precisely What You Implemented

- **Transport:** raw TCP via `socket.socket(AF_INET, SOCK_STREAM)`. No UDP anywhere.
 - **Connection model:** one new TCP connection per chunk, plus one dedicated connection for the initial resume-bitmap query. Nothing is kept alive/reused across chunks.
 - **Port usage:** receiver binds to a user-chosen port (default `9999`); sender's default guess is `1205` — **these defaults don't match**, so a real transfer requires the operator to type the same port on both sides. Worth knowing this is a UX quirk, not a protocol requirement.
 - **Buffers:** `recv_exact` loops `sock.recv(n - len(data))` until exactly `n` bytes are collected — handles partial TCP reads correctly (a common bug source if done naively with a single `recv()` call).
 - **Chunking:** fixed 4MB (`DEFAULT_CHUNK_SIZE = 4 * 1024 * 1024`), last chunk truncated to the remainder via `chunk_size_for_id`.
 - **Concurrency, sender side:** `ThreadPoolExecutor(max_workers=min(8, total_chunks))` — bounded parallelism.
 - **Concurrency, receiver side:** unbounded — a new `threading.Thread` per accepted connection (`receive_connection`), with **no pool size cap**. **Inference/limitation:** a burst of connections (malicious or accidental) could spawn unbounded threads.
 - **Timeouts:** `client.settimeout(10)` per connection on the receiver; `server.settimeout(2)` on the accept loop for periodic idle checks.
 - **Retries:** chunk-level, 3 attempts with linear backoff (`0.5*attempt`), plus up to 3 resume rounds if chunks still fail after a full pass.
 - **Termination:** `client.close()` in a `finally` block after every connection; server shutdown on `KeyboardInterrupt` or explicit "n" to "receive another file?".
-

SECTION 5 — Security, Implemented vs Partial vs Not Implemented

Feature	Status	Detail
AES-256	✓ Implemented	<code>dkLen=32</code> in PBKDF2 → 32-byte = 256-bit key fed to <code>AES.new</code> .
AES-EAX mode	✓ Implemented	Chosen because it's an AEAD mode — gives you ciphertext <i>and</i> an authentication tag in one pass, so you don't need a separate MAC step.
PBKDF2	✓ Implemented	200,000 rounds, per-file random 32-byte salt, deriving the <i>session</i> key from the human password.
Per-chunk key derivation	✓ Implemented	HMAC-SHA256(session_key, "CNFT/1.0-chunk-key" + chunk_id) — your code comments call this "HKDF-style"; it's really just a keyed HMAC expand step, not full RFC 5869 HKDF (no separate extract/salt-on-the-key phase). Say this precisely if asked — don't claim "HKDF" as if it's the standard construction.
Nonce	✓ Implemented	Fresh random 16 bytes per chunk (<code>get_random_bytes(16)</code>), on top of an already-unique per-chunk key — belt and suspenders against nonce reuse.
Authentication tag / MAC	✓ Implemented	AES-EAX's built-in 16-byte tag, explicitly checked via <code>decrypt_and_verify</code> (raises on mismatch — you don't just decrypt and hope).
Challenge-response auth	✓ Implemented, but asymmetric	See detailed flow below — only the sender proves knowledge of the preset code to the receiver; the receiver never proves anything back to the sender.
Replay-attack prevention	✓ Implemented	Session ID + timestamp freshness window + fingerprint + "completed" session rejection, all in <code>transfer_security.py</code> .
Auth rate-limiting/brute-force protection	✓ Implemented	Per-IP, 5 failures/5 min → 10-min block.
Whole-file integrity	✓ Implemented	SHA-256 recomputed post-assembly, compared against sender's pre-computed hash.

Feature	Status	Detail
Confidentiality	✅ Implemented	AES-256-EAX per chunk.
Password-confirmation handshake before full send	⚠️ Overstated in README	Your README checks this off, but there's no dedicated small-encrypted-probe step. A wrong password isn't caught up front — it surfaces only when the receiver's <code>decrypt_and_verify</code> fails on the <i>first</i> chunk it processes, which (because of retries/threading) can still mean several chunk-connections are attempted before the failure becomes obvious. This is a real gap between the README claim and the code.
MSG_ERROR / <code>send_error</code> protocol path	❌ Not implemented in practice	Function exists, never called.
Encrypted-at-rest storage	❌ Not implemented	Files are decrypted and written to disk in the clear; only in-transit chunks are encrypted.
End-to-end key exchange (RSA/ECC)	❌ Not implemented	Preset code + password are shared out-of-band (typed by the operator), not exchanged cryptographically.
Authorization / access control	❌ Not implemented	Anyone with the preset code can request any file the receiver is willing to serve — no per-user accounts or allow-lists.
Transport-layer encryption (TLS)	❌ Not implemented	The initial challenge/response bytes and the CNFT headers themselves travel over plain TCP (though the <i>file content</i> itself is AES-encrypted before it ever hits the socket).

The auth flow, in detail (important — get this exactly right):

1. Receiver generates `challenge` (32 random bytes), sends it.
2. Sender computes `HMAC(preset_code, challenge)`, sends it back.
3. Receiver verifies with `hmac.compare_digest` (constant-time comparison — correct, avoids timing attacks). If it fails: send `AUTH_FAIL`, close.
4. Receiver generates a **second** random value, `counter_challenge`, sends it.
5. Sender computes `HMAC(preset_code, counter_challenge)`, sends it back.
6. Receiver verifies again; if both pass, sends `AUTH_SUCCESS`.

Inference: despite the name "counter-challenge" (which suggests mutual auth), both steps are the *receiver* validating the *sender* — the sender never issues its own challenge for the receiver to answer, and the sender's only signal that the receiver is legitimate is receiving `AUTH_SUCCESS` at the end (which any script holding the preset code could send). So this is **one-directional authentication done twice**, not mutual authentication. If an interviewer asks "is this mutual auth?" — the honest answer is no, and that's a legitimate, explainable design tradeoff (the shared preset code is really the trust anchor, not the protocol shape).

SECTION 6 — File Transfer Workflow, End to End

1. Operator runs `work_sender.py`, enters the shared `preset_code`, receiver IP/port, picks a file via the tkinter dialog, and types a **file password** (separate from the preset code).
2. `send_file` computes the SHA-256 of the whole file (`compute_file_hash`, with a `tqdm` bar) and generates a random 32-byte salt.
3. `derive_session_key(password, salt)` → PBKDF2 → 256-bit session key (never sent over the wire — only the salt is).
4. `request_resume_bitmap` opens one connection: auth handshake → `send_header(MSG_RESUME_QUERY)` → `send_transfer_metadata(...)` → receiver replies `MSG_RESUME_BITMAP` + the bitmap of chunks it already has (all zero on a fresh transfer).
5. Receiver, on its side (`handle_resume_query`): validates the session via `register_or_reject_session`, creates/loads the manifest (`ensure_manifest`), prompts the human operator for the **same file password**, derives the same session key from the salt it just received, and replies with its bitmap.
6. Back on the sender: `list_missing_chunks(bitmap, total_chunks)` gives the work list. All missing chunks are submitted to a `ThreadPoolExecutor`.
7. For each chunk (`send_chunk`): open a fresh connection → auth again → send `MSG_DATA` + metadata + `chunk_id` → derive `chunk_key = derive_chunk_key(session_key, chunk_id)` → read that byte range from disk → `AES.new(chunk_key, AES.MODE_EAX, nonce)` → `encrypt_and_digest` → send `nonce`, `tag`, `ciphertext` → wait for `MSG_ACK`.
8. Receiver (`handle_chunk_data`): validates `chunk_id` is in range, decrypts+verifies the tag, writes the plaintext at the correct byte offset in the pre-allocated part file, marks the bit in the manifest bitmap, saves the manifest.
9. If a chunk's connection drops or times out, the sender retries up to 3 times with backoff; if it still fails, the whole batch finishes, the sender re-queries the bitmap, and does another resume round (up to 3 rounds total).
10. Once every bit is set (`bitmap_all_set`), the receiver atomically renames the part file to its final name (`os.replace`), marks the manifest complete, marks the session completed

(closing the replay window), and recomputes the SHA-256 to confirm end-to-end integrity.

11. Sender's progress bar completes; both sides print success and offer to send/receive another file.

SECTION 7 — Design Decisions

Why TCP instead of UDP? Decision: TCP for all transport. Why: file transfer needs guaranteed, ordered, complete delivery of every byte — TCP gives you that natively (retransmission, ordering, flow control) so you don't have to reimplement reliability logic on top of UDP. Trade-off: more per-packet overhead and handshake cost, which your "new connection per chunk" design multiplies further. Alternative considered (inference, not in code): QUIC/UDP-based reliable transport would reduce handshake overhead but is a much bigger engineering lift for a course project.

Why AES instead of RSA for the file data? Decision: symmetric AES-256-EAX for all chunk encryption. Why: symmetric ciphers are orders of magnitude faster for bulk data, and you already have a shared secret (the password) to derive a key from — RSA/asymmetric crypto is for solving the *key exchange* problem when you don't have a pre-shared secret, which isn't your situation (the password is shared out-of-band). Trade-off: the password itself has to reach both parties some other way (typed manually) — there's no cryptographic key-exchange protocol here.

Why PBKDF2? Decision: PBKDF2 with 200,000 rounds to turn a human password into a key. Why: passwords have low entropy; PBKDF2's deliberate slowness (adjustable round count) makes offline brute-force/dictionary attacks on a captured salt+ciphertext much more expensive. Alternative (inference): Argon2/scrypt are considered stronger against GPU/ASIC attacks today, but PBKDF2 is simple, in `pycryptodome` already, and adequate for a course project's threat model.

Why HMAC-based per-chunk keys instead of one key for the whole file? Decision: derive a unique key per chunk from the session key. Why: limits the "blast radius" if any single chunk's key or nonce were ever compromised, and structurally prevents nonce+key reuse across chunks even with random nonces. Trade-off: extra HMAC computation per chunk (cheap in practice).

Why challenge-response (rather than just sending the preset code)? Decision: HMAC challenge-response instead of transmitting the preset code itself. Why: the shared secret never crosses the wire, even in a form an eavesdropper could directly reuse. Trade-off (as covered in Section 5): your implementation is one-directional-done-twice, not true mutual auth — a design gap worth acknowledging rather than glossing over.

Why concurrent/chunked transfers with a new connection per chunk? Decision: parallel chunk sends via a thread pool, each on its own socket. Why: enables real throughput gains on larger files and makes resume trivial — a failed chunk is just "a chunk that didn't get an ACK," retried independently, no shared connection state to untangle. Trade-off: repeated TCP + auth handshake cost per chunk; with 4MB chunks this is a meaningful chunk-count-dependent overhead, and it's why chunk size (and thread count) are worth discussing as tunable parameters.

Why bitmap-based resume instead of byte-offset resume? Decision: a bit per chunk rather than "resume from byte N." Why: supports out-of-order and parallel chunk delivery (essential once you added concurrency) — a simple offset can't represent "chunks 3, 7, and 12 are done but 4-6 aren't."

SECTION 8 — Interview Questions (grouped by difficulty)

Easy

1. What port does your receiver listen on by default, and can it be changed?
2. Why did you choose TCP over UDP?
3. What does "peer-to-peer" mean in your project, as opposed to client-server with a relay?
4. What library provides your AES implementation?
5. What's a nonce, and where does yours come from?
6. What happens if the receiver isn't running when the sender tries to connect?
7. How does your receiver auto-detect its own IP address?
8. What's stored in the `.p2p_resume` folder?
9. What's the default chunk size, and why that number?
10. How do you compute the file's integrity hash?
11. What Python threading primitive do you use to protect shared state?
12. Where does the file password come from, and is it sent over the network?

Medium 13. Walk me through your authentication handshake step by step. 14. Why do you derive a new key per chunk instead of reusing the session key directly? 15. What would happen if two chunks reused the same nonce and key? 16. How does your bitmap-based resume actually work on disk? 17. What guarantees does AES-EAX give you that AES-CBC wouldn't? 18. Why do you open a new socket connection for every chunk instead of reusing one? 19. How do you prevent a stale/replayed session from being accepted twice? 20. What's the purpose of the `session_salt` field in your metadata? 21. How does your code detect and reject a tampered chunk? 22. What happens on your receiver if two different senders try to auth at the same time? 23. Why is `hmac.compare_digest` used instead of `==` for comparing

HMAC outputs? 24. Explain your rate-limiting logic for failed authentication attempts. 25. How is your `(transfer_id)` computed, and what happens if I rename the file and resend it?

Hard 26. Is your challenge-response protocol actually mutual authentication? Defend your answer. 27. Where exactly in your code is the critical section that could race if the lock were removed? 28. What happens, precisely, if someone sends the wrong password — where does the failure actually surface? 29. Your `(MSG_ERROR)`/`(send_error)` code exists but isn't called — what's missing to wire it in, and what would you replace your current generic exception handling with? 30. How would a network-level attacker distinguish this protocol's handshake bytes from random noise, and does that matter for your threat model? 31. What happens to in-flight chunk threads if the receiver process is killed mid-transfer? 32. Explain exactly why the per-chunk key derivation is described as "HKDF-style" rather than actual HKDF, and whether that distinction matters cryptographically. 33. Could two legitimate senders behind the same NAT/IP get auth-throttled because of each other's failures? Why? 34. Walk through what happens if the same `(transfer_id)` shows up with a different `(session_id)` mid-transfer. 35. Why is the decrypt+write step inside the receiver's global lock, and what would break if you moved it outside?

Very Hard 36. Design a variant of your protocol that achieves genuine mutual authentication without adding a third round trip. 37. Your resume identity is tied to `(abspath + mtime + size)`. Propose a content-addressed alternative and its tradeoffs (e.g., cost of hashing large files up front vs. resumability across machines). 38. How would you extend this to support multiple concurrent *different* files between the same sender/receiver pair, given your current global state variables in `(work_rec.py)`? 39. Your receiver spawns an unbounded thread per connection — sketch a bounded-worker-pool redesign and what backpressure would look like. 40. If you wanted forward secrecy for stored (encrypted-at-rest) history of transfers, where would you need to change the key derivation, and what would break? 41. Suppose an attacker can observe encrypted chunk sizes and timing but not content — what could they infer about the file, and how would you mitigate it (e.g., padding)? 42. How would you formally verify your challenge-response protocol (e.g., using a tool like ProVerif or Tamarin) — what's the trust model you'd state? 43. Propose a migration path from your current pre-shared-preset-code model to a public-key-based one without breaking backward compatibility with old clients. 44. Your PBKDF2 round count is fixed at 200,000 — how would you make this adaptively tuned to hardware, and why does that matter operationally? 45. What's the actual max useful concurrency for your sender's thread pool given "open a new TCP+auth handshake per chunk," and how would you measure it?

Systems/behavioral crossover 46. What was the hardest bug you hit building the resume logic, and how did you find it? 47. If you had to demo this live and the network failed mid-transfer, what would you show the interviewer to prove resilience? 48. What's one design decision you'd defend strongly, and one you now think was wrong? 49. How did you test the

replay-protection logic — do you have any tests? 50. What's the single most impressive line of code in this project, and why?

SECTION 9 — Follow-Ups (representative sample — the pattern is the same for the rest)

- **Q13 (walk through auth)** → Follow-up: "So is the sender authenticating the receiver, or only the reverse?" → Follow-up: "What stops a man-in-the-middle from relaying the challenge?" (Honest answer: **nothing** — there's no channel binding; this is a real gap, not covered by your implementation.)
 - **Q18 (new socket per chunk)** → Follow-up: "What's the throughput cost of that, roughly?" → Follow-up: "Would connection pooling break your retry/resume model?"
 - **Q26 (mutual auth?)** → Follow-up: "How would you fix it with one more round trip?" → Follow-up: "Does TLS client-cert auth solve this more simply?"
 - **Q28 (wrong password)** → Follow-up: "So a wrong password currently looks identical to a corrupted/tampered chunk from the receiver's point of view — how would you disambiguate them in logs?"
 - **Q35 (lock placement)** → Follow-up: "So decrypting the chunk of thread A blocks thread B's decrypt entirely — is that a bottleneck on multi-core hardware?" → Follow-up: "How would you shard the lock per-transfer instead of globally?"
-

SECTION 10 — Code Defense: Key Functions

`derive_chunk_key`

- Purpose: turn one session key into many independent per-chunk keys.
- Logic: `HMAC-SHA256(session_key, b"CNFT/1.0-chunk-key" + struct.pack("!I", chunk_id))`.
- Complexity: $O(1)$ per call, negligible cost relative to the AES work itself.
- Likely question: "Why HMAC and not just `session_key XOR chunk_id`?" — answer: HMAC gives you a cryptographically one-way, unpredictable output; XOR would leak structure (attacker who learns one chunk key could derive others).
- Possible improvement: use real HKDF (`Crypto.Protocol.KDF.HKDF`) for a textbook-correct extract-then-expand construction.

`handle_chunk_data`

- Purpose: receive, decrypt, verify, and persist one chunk.
- Logic: validates `chunk_id` range → derives chunk key → (inside lock) checks for a duplicate chunk → decrypts+verifies tag → writes at offset → updates bitmap → finalizes if complete.

- Complexity: dominated by AES-EAX decrypt, $O(\text{chunk_size})$.
- Likely question: "What if `chunk_id >= total_chunks`?" — you already guard this (`raise ValueError`).
- Potential bug (inference): if two threads race to be the *first* to complete the final chunk, the `bitmap_all_set` check and `os.replace` happen inside the lock, so this should be safe — but it's worth tracing aloud in the interview to show you understand why it's safe, not just that it "happens to work."
- Improvement: separate the crypto/decrypt work from the lock-protected manifest section to allow real parallel decryption.

`register_or_reject_session`

- Purpose: the replay-protection gate.
- Logic: prune expired *completed* sessions → check timestamp freshness → check fingerprint on session reuse → reject completed sessions.
- Complexity: $O(\text{number of cached sessions})$ due to the prune loop over the whole cache dict each call — fine at small scale, would need indexing/TTL-based expiry at larger scale.
- Likely question: "What's your freshness window and why 15 minutes?" — good chance to discuss clock-skew tolerance vs. replay-window size trade-off.

`send_chunk`

- Purpose: the sender's unit of work — connect, auth, send one encrypted chunk, await ACK, retry on failure.
 - Logic: `for attempt in range(1, MAX_RETRIES+1)`, catches connection-class exceptions for retry, breaks immediately on anything else.
 - Potential bug: a non-connection exception (e.g., a bug in your own encryption code) silently "breaks" and returns 0 — masking the real cause in the chunk's failure list. Good to know if asked about failure visibility.
-

SECTION 11 — Failure Scenarios

Scenario	Behavior in your code
Wrong password	Session key derived incorrectly → AES-EAX tag verification fails on the receiver → <code>decrypt_and_verify</code> raises → caught by the generic <code>except Exception</code> in <code>receive_connection</code> → connection closed, chunk marked failed on sender, retried (uselessly, since the password won't change) until <code>MAX_RETRIES</code> / <code>MAX_RESUME_ROUNDS</code> are exhausted.
Tampered ciphertext	Same path — AES-EAX tag check fails, chunk rejected, connection closed.
Dropped connection mid-chunk	Caught as <code>ConnectionError</code> / <code>BrokenPipeError</code> / <code>TimeoutError</code> in <code>send_chunk</code> , retried up to 3 times with backoff.
Corrupted packet (bit flip in transit)	Would almost certainly fail the AES-EAX tag check (this is exactly what AEAD is for) — treated like tampering.
Duplicate packets/chunks	<code>handle_chunk_data</code> checks <code>bitmap_has_chunk</code> before processing and just re-ACKs without re-writing if already received.
Client disconnects unexpectedly	Receiver's per-connection thread hits an exception on <code>recv</code> , logs it, closes the socket; the overall file transfer isn't aborted — it just waits (up to <code>idle_grace=8s</code> after last activity) before giving up on that attempt, with the manifest saved for a future resume.
Large files	Handled by chunking; no explicit max-file-size guard exists in the code (not present — worth stating explicitly if asked).
Multiple simultaneous transfers (same sender/receiver pair)	Not supported cleanly — <code>work_rec.py</code> has one set of global state variables (<code>password_global</code> , <code>active_transfer_id</code> , etc.) reset per file via <code>reset_state()</code> ; this is inherently single-transfer-at-a-time per receiver process.
Insufficient permissions (can't write to <code>save_dir</code>)	Not explicitly handled — would surface as an unhandled <code>OSError</code> / <code>PermissionError</code> bubbling up through the generic exception catch, likely just logged and the connection dropped.
Disk full	Not explicitly handled — <code>ensure_part_file</code> 's <code>truncate()</code> or the chunk <code>write()</code> would raise <code>OSError</code> , caught only by the same generic handler.

SECTION 12 — Limitations (don't hide these — own them)

- **No transport-layer encryption of the handshake bytes** — the challenge/response and CNFT headers travel in plaintext (only file *content* is encrypted).
- **Auth is not mutual** — receiver validates sender twice; sender has no cryptographic proof the receiver is legitimate beyond trusting the shared preset code.
- **Unbounded thread creation on the receiver** for incoming connections — no cap, no pool, potential resource exhaustion under connection floods.
- **Global lock serializes all chunk decryption**, capping real receive-side throughput regardless of core count.
- **Single-transfer state model** — receiver globals mean one file transfer at a time per process; can't handle two unrelated concurrent transfers cleanly.
- **Resume identity tied to sender-side path+mtime** — doesn't survive a file being copied, moved, or touched.
- **No `.env`/secrets management** — preset code and password are typed interactively each run (your own `note.txt` already flags this).
- **No CLI flags** — everything is `input()`-driven, which blocks automation/scripting.
- **No max file size or disk-space pre-check** — large files or a full disk fail ungracefully via generic exception handling.
- **Windows/macOS/Linux firewall handling is inconsistent** — cross-platform IP *detection* exists (`receiver_detection.py`), but nothing in these files auto-opens a firewall port on any OS.
- **`MSG_ERROR` protocol path is dead code** — errors aren't communicated to the peer in a structured way; the peer just sees a closed connection.
- **PBKDF2 round count is fixed**, not adaptively tuned to the deployment hardware.

SECTION 13 — Improvements (if given another month)

Security

- Add real mutual authentication (e.g., both sides issue and answer a challenge, or move to mutual TLS).
- Replace the "HKDF-style" HMAC expand with `Crypto.Protocol.KDF.HKDF` proper.
- Wire `MSG_ERROR` into the actual error paths for structured, auditable failure reporting instead of only console prints.
- Move preset code / secrets into a `.env` file (already flagged in your own notes).

Networking

- Persistent connection reuse (keep-alive) for chunk streaming instead of reconnecting per chunk, cutting handshake overhead.
- Bound the receiver's thread pool with a `ThreadPoolExecutor` or `asyncio` instead of unbounded `threading.Thread` per connection.
- Configurable/negotiated chunk size based on measured network conditions.

Performance

- Move chunk decryption outside the global lock; use a per-transfer (or per-chunk-range) lock instead of one global lock.
- Benchmark and tune thread-pool size vs. chunk size for actual measured throughput rather than the current fixed values (8 threads, 4MB chunks).

Code quality

- Add `argparse`/CLI flags for automation.
- Add unit tests for KDF consistency, bitmap logic, and metadata round-tripping (none currently exist in the provided files).
- Replace `print()` statements with the `logging` module (INFO/WARN/ERROR levels), matching your own README roadmap.

Production-readiness

- Content-addressed resume identity (hash-based, not path/mtime-based) so resumes survive file moves/copies.
- Explicit disk-space and file-size preflight checks with clear error messages.
- Multi-transfer support on the receiver (per-transfer state instead of global module state).

SECTION 14 — CS Concepts to Have Cold

- TCP: three-way handshake, why it guarantees ordered/reliable delivery, `recv()` partial-read behavior (why `recv_exact` exists).
- Sockets: blocking I/O, `socket.timeout`, `SO_REUSEADDR`, listen backlog.
- Threads vs. processes: why you used threads (shared memory for the manifest/bitmap via one lock) rather than separate processes.
- Race conditions & critical sections: why the global `lock` exists around manifest read-modify-write.
- Symmetric vs. asymmetric encryption: why AES here, and where asymmetric crypto *would* fit (key exchange) if you added it later.
- AES modes: EAX as an AEAD mode vs. CBC (no built-in integrity) vs. GCM (a related AEAD alternative).

- Hash functions: SHA-256 properties (one-way, collision resistance) as used for both integrity and transfer IDs.
 - HMAC: keyed hash for authentication and for chunk-key derivation — different from a plain hash.
 - KDFs: PBKDF2's deliberate slowness against brute force; the difference between a KDF and a MAC.
 - Nonces: why uniqueness (not secrecy) is the property that matters for AES-EAX/GCM-style modes.
 - Replay attacks: why a timestamp + session ID + "completed" tracking defeats them.
 - Authentication vs. authorization vs. confidentiality vs. integrity: map each term to the specific mechanism in your code that provides it (or admit which ones you don't provide).
 - Bitmaps as a compact resumability data structure.
 - Idempotency: why re-processing an already-received chunk (`bitmap_has_chunk` check) is safe.
-

SECTION 15 — Rapid Fire (60 one-liners)

1. What port does the receiver default to? *9999*.
2. What port does the sender default to? *1205 — note the mismatch*.
3. Default chunk size? *4MB*.
4. PBKDF2 round count? *200,000*.
5. Key length? *32 bytes / 256 bits*.
6. AES mode used? *EAX*.
7. Nonce length? *16 bytes*.
8. Tag length? *16 bytes*.
9. Session ID length? *32 hex chars (16 random bytes)*.
10. Auth failure threshold before block? *5*.
11. Auth failure window? *5 minutes*.
12. Auth block duration? *10 minutes*.
13. Session freshness window? *15 minutes*.
14. Completed-session retention? *24 hours*.
15. Max chunk send retries? *3*.
16. Max resume rounds? *3*.
17. Max sender thread-pool size? *8 (or fewer if fewer chunks)*.
18. Which hash algorithm verifies whole-file integrity? *SHA-256*.

19. Which module derives the session key? `transfer_resume.py`.
20. Which module enforces replay protection? `transfer_security.py`.
21. Which module defines the wire protocol header format? `cnft_protocol.py`.
22. Is TLS used anywhere? *No*.
23. Is the preset code ever sent in plaintext? *No — only HMAC responses to random challenges.*
24. Is the file password ever sent over the network? *No — only the salt is sent; the key is derived independently on both sides.*
25. What triggers a resume round? *One or more chunks failing after all retries.*
26. What data structure tracks received chunks? *A bitmap (bytearray, 1 bit per chunk).*
27. What happens when a duplicate chunk arrives? *It's detected via the bitmap and just re-ACKed without rewriting.*
28. Where is the manifest stored? `.p2p_resume/<transfer_id>.json`.
29. Where is the partial file stored? `.p2p_resume/<transfer_id>.part`.
30. How is the final file named? `received_<filename>` in the save directory.
31. What renames the part file to final? `os.replace`.
32. Is that rename atomic? Yes, `os.replace` is atomic on the same filesystem.
33. What identifies a transfer for resume purposes? *SHA-256 of path+size+mtime+chunk_size, truncated to 32 hex chars.*
34. Does moving the source file break resume? Yes — new `transfer_id`.
35. What Python crypto library is used? `pycryptodome`.
36. Which class handles AES encryption? `Crypto.Cipher.AES`.
37. Which function generates cryptographically secure random bytes? `Crypto.Random.get_random_bytes`.
38. What Python module provides constant-time comparison? `hmac.compare_digest`.
39. What's the risk of using `==` instead? *Timing attack — a naive comparison leaks how many leading bytes matched.*
40. Is there a `.env` file for secrets currently? *No — flagged as a TODO in your own notes.*
41. Are there CLI flags? *No — everything is interactive* `input()`.
42. Is logging used, or print statements? *Print statements.*
43. What GUI library is used for file/folder pickers? `tkinter`, run in an isolated subprocess.
44. Why run tkinter in a subprocess? *To avoid Tcl/thread conflicts with the main process.*
45. How does the receiver detect its own LAN IP? *OS-specific commands* `(ipconfig/ifconfig/hostname -I)` with a socket-based fallback.
46. Is IPv6 supported? *Not present in the code — IPv4* `(AF_INET)` only.

47. What socket option avoids "address already in use" errors on restart? `SO_REUSEADDR`.
 48. What's the accept-loop backlog size? 50.
 49. How long does the receiver wait before giving up on an idle transfer? *8 seconds after last activity.*
 50. Is there authentication on the resume-bitmap-query connection too, or just chunk sends? *Both — every connection re-authenticates.*
 51. Does the receiver validate `chunk_id` bounds? *Yes — rejects if `>= total_chunks`.*
 52. What happens on a malformed CNFT header? `ValueError` is raised in `recv_header`.
 53. What's the max header line length allowed? 64 bytes.
 54. Is `MSG_ERROR` ever actually sent? *No — dead code in the current flow.*
 55. Is `MSG_AUTH` ever actually sent as a CNFT header? *No — auth happens via raw bytes before any header.*
 56. Does the sender verify the receiver's identity? *No — only the receiver verifies the sender.*
 57. What happens if two threads try to write the final chunk at once? *Serialized by the global lock — safe, but not parallel.*
 58. Is decryption itself parallelized across chunks on the receiver? *No — it happens inside the global lock, effectively serialized.*
 59. What's the biggest unaddressed concurrency risk? *Unbounded thread creation per incoming connection on the receiver.*
 60. If you had to name one thing to fix first, what would it be? *(Your call — but "mutual auth" or "wire up MSG_ERROR" are both honest, defensible answers grounded in the code.)*
-

SECTION 16 — Resume Bullet Defense

Since I don't have your actual resume text, here's how to defend the *likely* bullet points for a project like this — adjust wording to match what's really on your resume:

- **"Built a peer-to-peer encrypted file transfer system in Python"** → Justified. `work_sender.py`/`work_rec.py` connect directly, no relay. Files: both work files.
- **"Implemented AES-256 encryption with authenticated encryption (EAX mode)"** → Justified, and technically precise. Files: `transfer_resume.py` (`derive_session_key`, `derive_chunk_key`), `work_rec.py`/`work_sender.py` (`AES.new(..., AES.MODE_EAX, ...)`).
- **"Designed a custom application-layer protocol (CNFT/1.0)"** → **Be careful with this wording.** Technically true that you defined header framing and a version string, but if your resume implies a *complete* message-type protocol, be ready to admit only `DATA`/`ACK`/`RESUME_QUERY`/`RESUME_BITMAP` are actually wired in — `AUTH`/`METADATA`/`ERROR` are defined but unused. Say "a lightweight framing protocol I extended as needed," not "a fully implemented multi-message protocol."

- **"Implemented resumable/interrupted-transfer support via chunk-level bitmaps"** → Fully justified. Files: `transfer_resume.py` (bitmap functions), `work_rec.py`/`work_sender.py` (resume-round logic).
 - **"Added replay-attack protection and rate-limited authentication"** → Fully justified. File: `transfer_security.py`.
 - **"Supported concurrent multi-threaded file transfer"** → Justified for the *sender* (`ThreadPoolExecutor`). Slightly overstated if implied for the receiver's *decryption* work, since that's serialized under one lock — be ready to clarify "concurrent connection handling" vs. "concurrent decryption throughput."
 - **"Prevented brute-force/wrong-password waste via handshake"** → **This is the one most likely to get you in trouble if worded strongly.** The README claims a dedicated password-confirmation step; the code doesn't have one distinct from the preset-code auth. If this is on your resume, soften it to "detected authentication/decryption failures via AEAD tag verification" rather than "pre-validated passwords before transfer."
-

SECTION 17 — Final Cheat Sheet

Architecture (text diagram)

Sender	Receiver
-----	-----
pick file, enter preset_code + file password	
compute SHA-256(file), generate salt	
derive session_key = PBKDF2(password, salt, 200k)	
	listen on port, accept
loop	
--- new connection ---	
auth: HMAC challenge/response (preset_code) -----> verify (2x, receiver-driven)	
send MSG_RESUME_QUERY + metadata -----> register session, load/create manifest	
	prompt password, derive
session_key	
<----- MSG_RESUME_BITMAP (current bitmap)	
compute missing chunks	
for each missing chunk (thread pool, up to 8):	
--- new connection, re-auth ---	
chunk_key = HMAC(session_key, "chunk-key"+id)	
nonce = random(16), encrypt+tag (AES-EAX)	
send MSG_DATA + metadata + chunk_id + nonce+tag+ct -> decrypt+verify tag, write at offset	
	mark bitmap bit, save
manifest	
<----- MSG_ACK	
retry failed chunks (3x) / re-query bitmap (3 rounds)	
	all bits set -> atomic
rename to final	
	recompute SHA-256,
compare, report	

Crypto choices at a glance: AES-256-EAX (confidentiality + integrity) · PBKDF2/200k (password→session key) · HMAC-SHA256 expand (session key→per-chunk key) · SHA-256 (whole-file integrity) · `secrets.token_hex` (session IDs) · `hmac.compare_digest` (timing-safe comparisons).

Strengths: genuine AEAD per chunk, real resumability with correct bitmap semantics, replay protection with fingerprint+freshness, IP-based auth throttling, cross-platform IP autodetection.

Limitations to state proactively: one-directional auth (not mutual), unbounded receiver threads, serialized decryption under a global lock, resume tied to path+mtime, no TLS on the handshake itself, `MSG_ERROR`/`MSG_AUTH`/`MSG_METADATA` partially dead code, no CLI/`.env`, single-transfer-at-a-time global state on the receiver.

Most likely questions: "Is your auth mutual?" / "Why AES-EAX?" / "What happens on wrong password?" / "Why a new connection per chunk?" / "What's actually serialized vs. parallel on the receiver?"

Mistakes to avoid when explaining this:

- Don't say "fully implemented custom protocol" — say which message types are actually wired in.
- Don't claim "mutual authentication" — it isn't, and claiming it invites a follow-up that exposes the gap.
- Don't claim receiver-side decryption is parallel — it's serialized by the lock.
- Don't claim there's a dedicated password-confirmation step — there isn't; the AEAD tag failure is what actually catches it.
- Do be ready to say "this isn't implemented" plainly rather than improvising an answer under pressure — interviewers respect precision over bluffing.

Discrepancies Found (Resume / README / Code) — flagged only, not resolved

1. **README checks off** "perform a small encrypted challenge-response before sending full file" to prevent wasted bandwidth on wrong passwords — **the code has no such dedicated step**; wrong passwords are only caught via AEAD tag failure during real chunk transfer.
2. `cnft_protocol.py` **defines** `MSG_AUTH`, `MSG_METADATA`, `MSG_ERROR` / `send_error` as part of the "formal protocol," but none of these are imported or used in `work_rec.py` / `work_sender.py` — the actual protocol in use is a subset.
3. **Sender's default port** (`1205`) **and receiver's default port** (`9999`) **don't match** — cosmetic but worth knowing if you're asked to demo defaults live.
4. **Per-chunk key derivation is called "HKDF-style" in code comments/README**, but the implementation is a single HMAC-SHA256 expand step, not the full two-phase HKDF (extract + expand) from RFC 5869.
5. **Your memory/notes mention a** `firewall_setup.py` **module and Windows Firewall auto-configuration** — this file is **not present** in the source directory dump you gave me for this analysis, so I can't verify or defend its implementation from the code alone; if it's on your resume or you expect questions on it, make sure it's actually in the version of the project you're bringing up tomorrow.